

Tema 3: Planificador de Trayectorias Profesionales

Proyecto Final de Inteligencia Artificial y Simulación

Rachel Mojena González C312

Junio 2026

1. Descripción del problema

Dado un conjunto de habilidades, cursos y restricciones de prerequisites, se busca construir una secuencia válida que permita alcanzar un objetivo profesional. Cada curso imparte una o más habilidades y exige otras como prerequisite; el usuario parte de un conjunto inicial de habilidades (que puede ser vacío) y dispone de un presupuesto limitado (o no) de semanas y de horas semanales. Una trayectoria es *válida* cuando: los prerequisites de cada curso están satisfechos en el momento de tomarlo, no se exceden las horas semanales ni las semanas totales, y al terminar se cubren todas las habilidades que define el rol profesional objetivo.

El sistema, además, utiliza un LLM en dos puntos: para *interpretar* objetivos expresados en lenguaje natural (“Quiero ser desarrollador backend...”) y para *evaluar* la calidad de las trayectorias propuestas (coherencia pedagógica, ausencia de redundancia, valor profesional, realismo). Entre las muchas trayectorias válidas posibles interesa la *mejor*: la que minimiza el costo (tiempo y dificultad) y maximiza la calidad estimada.

2. Modelado formal

Entidades. Sea S el conjunto de habilidades y C el de cursos. Cada curso $c \in C$ es una tupla $[c = (\text{pre}(c), \text{out}(c), d(c), h(c), \delta(c), p_0(c)),]$ donde $\text{pre}(c) \subseteq S$ son las habilidades que hay que tener *antes* de tomarlo y $\text{out}(c) \subseteq S$ las que se obtienen *al* aprobarlo. Los demás campos describen su esfuerzo y riesgo: duración $d(c)$ en semanas, carga $h(c)$ en horas por semana, dificultad $\delta(c) \in [0, 1]$ y probabilidad base de aprobarlo $p_0(c)$. Un rol objetivo se reduce a dos conjuntos de habilidades: las *requeridas* $G \subseteq S$ (obligatorias) y las *recomendadas* $G_{\text{rec}} \subseteq S$ (deseables). Por último, el estudiante se resume en su perfil $(H_0, W, H_{\text{sem}}, \rho)$: lo que ya sabe (H_0), cuántas semanas dispone (W), cuántas horas semanales puede dedicar (H_{sem}) y su tolerancia al riesgo (ρ).

Estados. El problema se modela como búsqueda en grafo. Un estado es

$$q = (\text{skills}(q), \text{seq}(q), w(q), \Delta(q)),$$

las habilidades acumuladas, la secuencia de cursos tomados, las semanas consumidas y la dificultad acumulada. El estado inicial es $q_0 = (H_0, \langle \rangle, 0, 0)$.

Transiciones. Tomar un curso lleva de un estado a otro. El curso c *puede* tomarse en q solo si se cumplen cuatro condiciones: ya se tienen sus prerequisites ($\text{pre}(c) \subseteq \text{skills}(q)$), no se ha tomado antes ($c \notin \text{seq}(q)$), cabe en la carga horaria ($h(c) \leq H_{\text{sem}}$) y no agota el presupuesto de tiempo ($w(q) + d(c) \leq W$). Al tomarlo, el nuevo estado suma las habilidades, el curso, las semanas y la dificultad correspondientes: $[\text{skills} \cup \text{out}(c), \text{seq} + \langle c \rangle, w + d(c), \Delta + \delta(c)]$.

Meta. Termina cuando el estado cubre todas las habilidades requeridas por el rol: q es meta si y solo si $G \subseteq \text{skills}(q)$.

Costo y heurística. El costo de camino combina tiempo, dificultad y número de cursos, con una bonificación por habilidades recomendadas cubiertas:

$$g(q) = w(q) + 2 \Delta(q) + 0,5 |\text{seq}(q)| - 0,5 |G_{\text{rec}} \cap \text{skills}(q)|.$$

La estimación que guía la búsqueda. Para no explorar a ciegas, A^* necesita una estimación del costo que falta consciente de los prerrequisitos: para cada habilidad que aún falta (incluido el encadenamiento de prerrequisitos que arrastra), suma el costo de su curso productor. Como alcanzar esa habilidad obliga, como mínimo, a tomar ese curso, la suma nunca sobrestima el costo real: es una cota inferior y, por tanto, una heurística admisible. Si alguna habilidad objetivo no la produce ningún curso, el objetivo es inalcanzable y $h = \infty$.

Objetivo de optimización. Hallar la secuencia válida de cursos que minimiza g . Para el *ranking* de planes alternativos se usa además una puntuación multiseñal, que incorpora las estimaciones de Monte Carlo y del evaluador LLM.

3. Conjuntos de datos

Se trabaja con dos datasets de igual esquema (JSON: `skills`, `courses`, `roles`, `instances`).

3.1. Dataset obtenido (dominio tecnología, curado)

Catálogo curado manualmente sobre carreras tecnológicas: **40 habilidades**, **40 cursos**, **13 roles** (Ingeniero de ML, Analista de datos, Científico de datos, Backend, DevOps, Full Stack, Robótica, ...) y **29 instancias** de prueba. Las habilidades se organizan en pistas coherentes (matemáticas, programación, ingeniería) con prerrequisitos encadenados. La inclusión de `course_python_basic` (sin prerrequisitos) garantiza que *toda* habilidad tenga al menos un curso productor y que cualquier objetivo sea alcanzable desde cero. Este dataset contiene los casos más profundos (p.ej. Robótica: plan de 11 cursos / 91 semanas), idóneos para estresar la búsqueda.

3.2. Dataset generado (dominio musical, sintético)

Generado por `instance_generator.py` a partir de una plantilla curada del dominio musical. Tamaño: **16 habilidades**, **16 cursos**, **5 roles** (Intérprete, Compositor, Productor, Profesor, Cantante) y **15 instancias**.

Proceso de generación. La plantilla define 16 habilidades en 5 pistas (Teoría, Instrumento, Composición, Producción, Canto) con un DAG de prerrequisitos *honesto* (sin ciclos, por niveles). Sobre esa estructura fija, un generador pseudoaleatorio sembrado muestrea los parámetros numéricos de cada curso como variables aleatorias escaladas por el nivel de la habilidad: la duración se extrae de un rango uniforme desplazado por el nivel, las horas semanales, la dificultad y la probabilidad base de aprobado como, de modo que un nivel más avanzado tiende a producir cursos más largos, más difíciles y con menor probabilidad de aprobar. Las parejas (perfil, rol) candidatas se recorren y solo se conservan las resolubles. El texto de cada objetivo se toma al azar de un banco de plantillas en lenguaje natural (“Quiero ser productor musical...”). Un **único sembrado** fija toda la cadena de extracciones aleatorias, haciendo el resultado enteramente reproducible.

Por qué cumple las condiciones del problema. (i) *Aciclicidad*: la plantilla por niveles impide ciclos de prerrequisitos, garantizando que exista orden topológico válido. (ii) *Alcanzabilidad*: toda habilidad es producible y las fundamentales tienen prerrequisitos vacíos, de modo que cualquier consulta es resoluble *desde cero* sin necesidad de habilidades iniciales. (iii) *Factibilidad*: los presupuestos de tiempo de las instancias se fijan holgados respecto al cierre de cada rol, por lo

que existe trayectoria válida (lo confirma el 100 % de éxito de A*/UCS, Tabla 2). (iv) *Coherencia semántica*: al partir de una plantilla curada, los nombres y prerrequisitos son interpretables por el LLM, que mapea consultas naturales al rol correcto, y puede evaluar los planes sin dificultad.

4. Diseño del algoritmo

El sistema implementa tres planificadores:

1. **Greedy**: en cada paso toma el curso aplicable que cubre más habilidades objetivo pendientes. Rápido pero no puede armar cadenas profundas de prerrequisitos y falla en objetivos avanzados.
2. **UCS**: Dijkstra puro, sin poda ni heurística. Como explota en objetivos profundos, lleva un tope de seguridad de nodos que garantiza terminación.
3. **A*** (planificador principal): usa g y la heurística admisible descritas arriba, y aplica una poda clave: solo expande cursos cuyo outcome pertenezca a (cierre de prerrequisitos) \ (habilidades actuales). Esto acota los estados a subconjuntos del cierre y descarta cursos irrelevantes sin perder optimalidad. Soporta k planes alternativos (top-3).

Canalización principal: A* + Monte Carlo + Evaluador LLM.

1. **A*** produce la trayectoria óptima (o k alternativas).
2. **Monte Carlo** estima la probabilidad de éxito del plan y su riesgo bajo incertidumbre.
3. **Evaluador LLM** puntúa el plan en 6 señales (*global_quality*, *goal_alignment*, *pedagogical_coherence*, *non_redundancy*, *professional_value*, *profile_realism*) y devuelve fortalezas y debilidades en lenguaje natural.

Estimación probabilística del riesgo (Monte Carlo). El plan de A* es óptimo en condiciones ideales, pero ningún estudiante aprueba todo curso al primer intento. Para cuantificar esa incertidumbre, cada plan se somete a un experimento aleatorio repetido N veces con un generador pseudoaleatorio sembrado. En cada realización se recorre la secuencia de cursos tratando el desempeño del estudiante como un proceso estocástico:

1. **Resultado por curso**: la probabilidad de aprobar parte de la base $p_0(c)$ y se descuenta por dificultad ($0,15\delta$), por sobrecarga de horas ($0,05$ por hora excedida) y por *fatiga* acumulada ($0,03f$), acotándose a $[0,05, 0,98]$. Un ensayo de Bernoulli con esa probabilidad decide aprobado/suspense.
2. **Duración como variable aleatoria**: la duración real de un curso es $d(c) \cdot \mathcal{N}(1, \sigma)$ con $\sigma = 0,10 + 0,05\delta$: un mismo curso no siempre toma lo mismo.
3. **Fatiga dinámica**: tras cada curso la fatiga evoluciona según la recurrencia $f \leftarrow 0,9f + \delta(c)$, de modo que encadenar cursos difíciles erosiona progresivamente la probabilidad de aprobar los siguientes.
4. **Reintentos y abandono**: un suspenso cuesta $2\times$ las semanas del curso (repetirlo); si se acumulan más de 2 suspensiones, la trayectoria se da por fallida.
5. **Éxito de la realización**: ocurre si el tiempo total acumulado no supera el presupuesto de semanas del perfil.

Al agregar las N realizaciones (ley de los grandes números) se obtienen estimadores estables: la probabilidad de éxito como proporción de realizaciones exitosas, con intervalo de confianza al 95 %; la distribución de semanas resumida en media y percentiles p10/p50/p90; y un **risk_score** = $(p_{90} - p_{50})/p_{50}$ que mide la penalización del peor escenario frente al caso típico. Esta señal es la que revela, por ejemplo, que el plan de Robótica –óptimo para A*– solo se completa en plazo el 15 % de las veces (Sección 7), información que el plan “en seco” oculta.

Puntuación multiseñal. La calidad final combina las señales:

$$\text{score} = w_{\text{req}} \text{req} + 0,15 \text{rec} + 0,25 \text{MC} + 0,20 \text{LLM} - 0,20 \text{tiempo} - 0,10 \text{dificultad}.$$

La cobertura requerida (w_{req}) vale 1,0 en todo plan válido y actúa como base; cuando MC o LLM no están activos, su peso se transfiere a w_{req} para mantener positiva la base.

5. Rol del LLM en el sistema

El LLM interviene en dos etapas y admite **dos vías** intercambiables:

- **Vía LLM real (Gemini):** `llm/client.py` envuelve `google-genai` (modelo `gemini-2.5-flash-lite`) y devuelve JSON; las respuestas se cachean por hash SHA256 del prompt, lo que aporta reproducibilidad y ahorro de llamadas.
- **Vía mock (sin API key):** `build_mock_goal_response` genera una respuesta determinista, agnóstica al dataset: empareja el rol por tokens y subcadenas sobre el rol. Permite ejecutar y testear todo el sistema sin conexión ni clave, y es la vía por defecto.

Interpretación. Aquí el LLM actúa como *traductor* entre el lenguaje natural del usuario y una especificación estructurada que el planificador pueda consumir. De una frase como “quiero ser científico de datos en un año dedicando 10 horas a la semana”, el modelo extrae los datos del problema: el rol profesional buscado, las habilidades que el usuario ya declara tener, las que quiere alcanzar y las restricciones de tiempo (semanas disponibles y horas por semana). Acompaña esa lectura con un grado de confianza, útil para señalar interpretaciones dudosas.

El punto clave del diseño es que esa lectura no se acepta a ciegas: todo lo que propone el modelo se contrasta contra el catálogo real de habilidades y roles del sistema. Las habilidades que no existen se descartan y el modelo nunca inventa datos: solo puede referirse a piezas que el catálogo ya contiene. Así se evita que una alucinación del LLM contamine la búsqueda posterior.

Este contraste plantea un dilema: ¿qué pasa cuando el objetivo del usuario no coincide con ningún rol predefinido? En lugar de forzar el encaje con un rol cercano, el diseño permite construir un rol personalizado: el LLM compone el objetivo combinando habilidades que sí existen en el catálogo, y el sistema lo trata como un rol más. De este modo el planificador funciona igual tanto para metas estándar como para metas personalizadas, sin necesidad de tener todos los roles posibles enumerados de antemano.

Evaluación. Una vez hallada la trayectoria, el LLM cumple un segundo papel, esta vez como revisor experto. La motivación es que el costo que optimiza la búsqueda mide tiempo, dificultad y número de cursos, pero es ciego a la calidad didáctica de la secuencia. Un plan puede ser óptimo en costo y, aun así, resultar pedagógicamente torpe: por ejemplo, colocar los fundamentos de matemáticas después de los cursos de machine learning que dependen de ellos. Ese tipo de defecto no se refleja en ningún número que la búsqueda sepa calcular.

El evaluador juzga el plan sobre varias dimensiones cualitativas complementarias: la coherencia pedagógica (que el orden tenga sentido, fundamentos antes que temas avanzados), la ausencia de redundancia (que no haya cursos que aporten habilidades ya cubiertas), la alineación con el objetivo (que lo cursado conduzca realmente al rol pedido), el valor profesional de las habilidades adquiridas y el realismo del perfil (que la carga sea asumible para el estudiante descrito). Cada dimensión recibe una puntuación, y se acompaña de una valoración global más una lista de fortalezas y debilidades redactadas en lenguaje natural.

Ese resultado cumple dos funciones en el diseño. Por un lado, alimenta la puntuación multiseñal que ordena los planes alternativos aportando un criterio que ningún cálculo puramente algorítmico capta. Por otro, ofrece al usuario una justificación legible de por qué su trayectoria es buena o dónde no, convirtiendo una secuencia de cursos en una recomendación argumentada.

6. Metodología experimental

El runner ejecuta cinco **variantes** sobre todas las instancias de un dataset y vuelca métricas a CSV; `plot_results.py` genera los gráficos comparativos.

- **Variantes:** `greedy`, `ucs`, `astar`, `astar_mc` (A^* + Monte Carlo) y `astar_mc_llm` (canalización completa).
- **Datasets:** el obtenido (`data/`, 29 instancias) y el generado (`data/generated/`, 15 instancias).
- **Parámetros:** Monte Carlo con $N = 500$ ejecuciones y semilla fija 42; tope de nodos por defecto.
- **Métricas:** tasa de éxito y validez, semanas y dificultad totales, cobertura, nodos expandidos y tamaño máximo de frontera (esfuerzo de búsqueda), tiempo de ejecución, probabilidad de éxito MC, calidad LLM y final score.

7. Resultados y análisis

7.1. Comparación entre variantes

Tabla 1: Dataset obtenido (dominio tecnológico, 29 instancias). Promedios por variante.

Variante	Éxito	Nodos exp.	Frontera máx.	Tiempo (s)	final_score
<code>greedy</code>	0.345	2.8	1.7	0.0002	0.235
<code>ucs</code>	0.448	2511.2	21525.7	0.401	0.294
<code>astar</code>	0.621	10.3	6.6	0.0009	0.400
<code>astar_mc</code>	0.621	10.3	6.6	0.0009	0.357
<code>astar_mc_llm</code>	0.621	10.3	6.6	0.0009	0.342

Tabla 2: Dataset generado (dominio musical, 15 instancias). Promedios por variante.

Variante	Éxito	Nodos exp.	Frontera máx.	Tiempo (s)	final_score
<code>greedy</code>	0.133	1.9	0.6	0.0001	0.081
<code>ucs</code>	1.000	58.0	106.5	0.0027	0.623
<code>astar</code>	1.000	8.2	3.4	0.0004	0.623
<code>astar_mc</code>	1.000	8.2	3.4	0.0004	0.602
<code>astar_mc_llm</code>	1.000	8.2	3.4	0.0004	0.576

En el dataset tecnológico (objetivos profundos), A^* es la única variante que resuelve la mayoría: 62 % de éxito frente al 34 % de `greedy` y el 45 % de UCS, con un costo de búsqueda ínfimo (~ 10 nodos). UCS, sin poda, choca contra el tope de 5000 nodos y hace crecer la frontera hasta 62mil en los casos difíciles. En el dataset musical (DAG poco profundo, sin casos esquina y tiempos holgados) todas las búsquedas completas tienen éxito, pero A^* sigue siendo mucho más eficiente que UCS (8,2 vs 58 nodos; frontera 3,4 vs 106). `Greedy` fracasa en ambos en cuanto el objetivo exige encadenar prerrequisitos.

Por qué MC y LLM “bajan” el score. En las dos tablas, `astar`, `astar_mc` y `astar_mc_llm` producen el mismo plan (mismas semanas, mismos cursos); sin embargo el **final_score** desciende al activar las señales. No es una regresión: con la fórmula de la Sección 4, al activar MC/LLM el peso que absorbía la cobertura requerida ($= 1,0$) se redistribuye hacia señales realistas (MC $\approx 0,72$, LLM $\approx 0,88$ en el dominio tecnológico). El score baja porque pasa a reflejar el riesgo real de completar el plan, información que el plan “en seco” ocultaba.

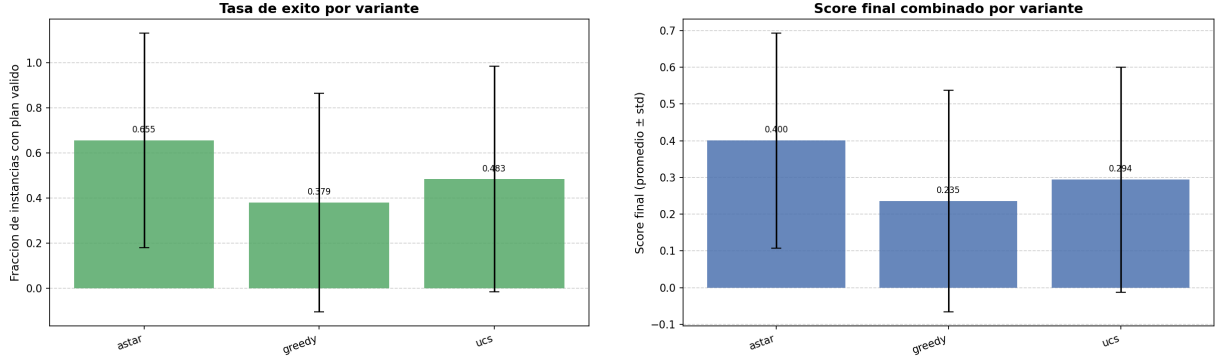


Figura 1: Dataset tecnológico: tasa de éxito (izq.) y `final_score` (der.) por variante. A* duplica la tasa de éxito de greedy y supera a UCS.

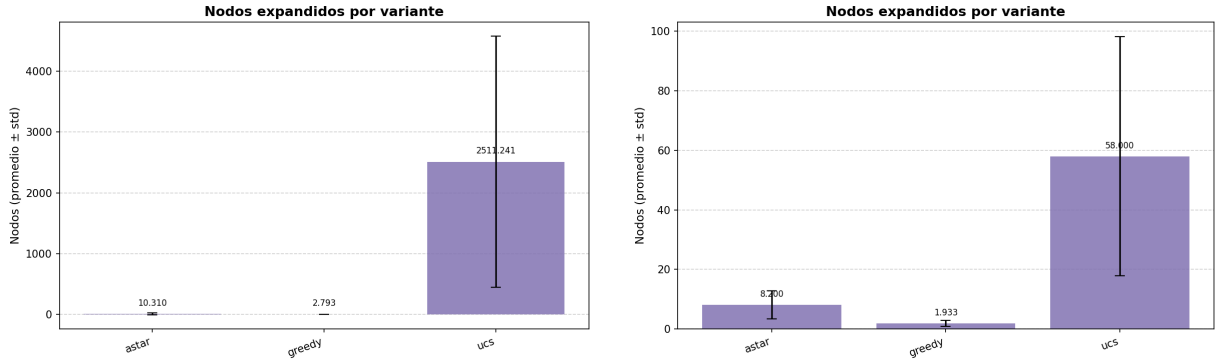


Figura 2: Nodos expandidos (escala muy distinta). Dataset tecnológico (izq.): UCS promedia ~ 2500 nodos frente a aproximadamente 10 de A*. Dataset musical (der.): 58 vs 8,2. La heurística + poda de A* reducen el esfuerzo en uno a tres órdenes de magnitud.

7.2. Caso 1 — Rol personalizado generado por el LLM (Frontend)

Consulta: “Quiero ser desarrollador frontend especializado en interfaces web; no me interesa el backend ni las bases de datos. Tengo 30 semanas y 8 horas por semana.”

El LLM real (Gemini) reconoce que *ningún* rol del catálogo encaja y sintetiza uno personalizado: `role_id="custom_desarrollador_frontend"`, `role_name="Desarrollador Frontend"`, con `target_skill_ids = {skill_html_css, skill_javascript_basic, skill_react_basic}`. Crucialmente respeta la preferencia “no backend ni bases de datos”: no añade habilidades fuera de ese alcance. A* resuelve en **4 nodos** expandidos un plan de **3 cursos / 14 semanas** (HTML/CSS $\rightarrow$ JavaScript $\rightarrow$ React). Monte Carlo estima **99.2 %** de éxito (IC 95 % [0,980, 0,997]) y el evaluador LLM otorga **0.9** de calidad global con `goal_alignment = 1,0`: “cubre los fundamentos antes de los frameworks”. `final_score = 0,591`. El caso muestra cómo el LLM extiende el sistema más allá del catálogo fijo de roles preservando las restricciones del usuario.

7.3. Caso 2 — Objetivo profundo resuelto eficientemente (Robótica)

Consulta: “Quiero ser ingeniero de robótica enfocado en percepción y control: arquitectura de computadoras, aprendizaje por refuerzo y matemáticas. Ya sé Python básico. Tengo 130 semanas y 12 horas por semana.”

Es el objetivo más profundo del dataset: requiere aprendizaje por refuerzo, álgebra lineal, cálculo y estadística avanzada, cuyo cierre de prerrequisitos encadena **11 cursos / 91 semanas**. Greedy *falla* (no arma la cadena) y UCS, sin poda, dispararía la frontera. A*, gracias a la heurística + poda por cierre, lo resuelve expandiendo solo **94 nodos** en $\sim 0,01$ s, con frontera máxima 55.

El evaluador LLM da **0.8** de calidad global (señala que cálculo y estadística avanzada llegan algo tarde respecto a los cursos de ML). Monte Carlo, en cambio, advierte el verdadero riesgo: con tantos cursos difíciles encadenados la probabilidad de completar en plazo es de solo **15.2 %** (semanas esperadas ≈ 88 , $p_{90} \approx 114$). Es el ejemplo canónico de las tres señales colaborando: A* halla el camino, el LLM valida la pedagogía y Monte Carlo cuantifica la dificultad de ejecutarlo.

8. Limitaciones y posibles mejoras

1. **Greedy** se mantiene solo como base comparativa; su fracaso en cadenas profundas está documentado y no se intentó reforzar.
2. **UCS** conserva un tope fijo de 5000 nodos; en objetivos muy grandes no garantiza encontrar el óptimo (se prioriza terminación sobre completitud), por diseño, para servir de línea base.
3. **Dependencia del LLM real** para roles a medida: el modo mock no sintetiza roles `custom_`, por lo que esa capacidad requiere clave de Gemini. Una mejora sería un sintetizador de roles offline basado en *embeddings* de habilidades.
4. **Calibración de Monte Carlo**: las probabilidades base y los factores de fatiga/sobrecarga son parámetros razonables pero no ajustados a datos reales; convendría calibrarlos con históricos de aprobados.

9. Interfaz interactiva

Como complemento a la línea de comandos, el sistema incluye una interfaz web ligera (construida con Gradio) que envuelve toda la canalización: el usuario escribe su objetivo en lenguaje natural –o lo elige de unos ejemplos predefinidos–, ajusta la configuración (proveedor del LLM, planificador, parámetros de Monte Carlo) en un panel lateral y obtiene el resultado organizado en pestañas (plan, puntuación con su desglose, riesgo probabilístico, evaluación cualitativa y ranking de alternativas).

Incorpora además un visor del grafo de prerequisites: un grafo dirigido donde cada nodo es un curso y una arista $A \rightarrow B$ indica que B requiere una habilidad que produce A , con un color distinto por nodo para seguir de un vistazo de dónde viene cada dependencia y los cursos del rol objetivo resaltados. Esta vista hace tangible la estructura que la búsqueda explora internamente y ayuda a entender por qué ciertos objetivos exigen cadenas largas de cursos.